Neural Networks - Lecture 3

Optimization Methods

Alexandru Sorici

UPB

Table of contents

1. Introduction
2. Gradient Descent
3. Second Order Optimization
4. Regularization
5. Stochastic Gradient Descent
6. Gradient Descent Optimization Algorithms
7. Optimization - other practical suggestions
8. References

Intro

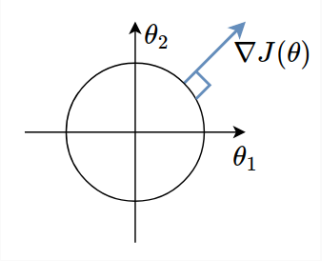
Recap - partial derivatives and gradient

Consider a two variable function $f(\boldsymbol{\theta}) = f(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$

$$\frac{\partial f(\theta_1, \theta_2)}{\partial \theta_1} = \lim_{\Delta\theta_1 \to 0} \frac{f(\theta_1 + \Delta\theta_1, \theta_2) - f(\theta_1, \theta_2)}{\Delta\theta_1}$$

$$\frac{\partial f(\theta)}{\partial \theta_1} = 2\theta_1 \qquad \frac{\partial f(\theta)}{\partial \theta_2} = 2\theta_2$$

$$\nabla f(\boldsymbol{\theta}) = \begin{bmatrix} 2\theta_1 \\ 2\theta_2 \end{bmatrix}$$



Recap - second order derivatives - Hessian

Hessian = square matrix of second-order partial derivatives

$$\frac{\partial}{\partial \theta_1}\left(\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_1}\right) = \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_1^2} = 2$$

$$\frac{\partial}{\partial \theta_2}\left(\frac{\partial f(\boldsymbol{\theta})}{\partial \theta_2}\right) = \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_2^2} = 2$$

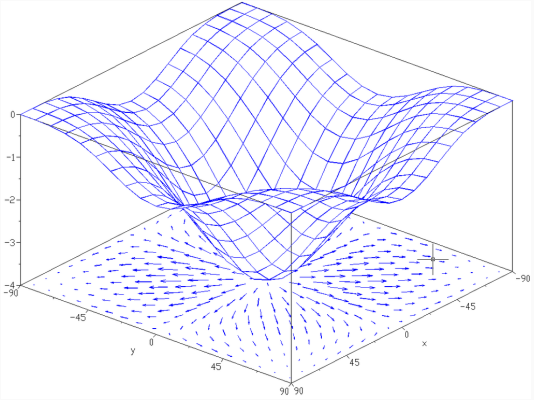
$$\frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_1 \theta_2} = \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_2 \theta_1} = 0$$

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Recap - General Case - gradient vector

$\boldsymbol{\theta}$ - a d-dimensional vector
$f(\boldsymbol{\theta})$ - scalar-valued function
Gradient vector of $f(\bullet)$ with respect to $\boldsymbol{\theta}$:

$$\nabla f(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_1} \\ \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_2} \\ \vdots \\ \frac{\partial f(\boldsymbol{\theta})}{\partial \theta_d} \end{bmatrix}$$



Recap - General Case - Hessian matrix

$\boldsymbol{\theta}$ - a d-dimensional vector

$f(\boldsymbol{\theta})$ - scalar-valued function

The Hessian matrix of $f(\bullet)$ with respect to $\boldsymbol{\theta}$ (written as $\nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$ or $\boldsymbol{H}$) is a $d \times d$ matrix of second-order partial derivatives:

$$H = \nabla^2_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = \begin{bmatrix} \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_1^2} & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_1 \theta_2} & \cdots & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_1 \theta_d} \\ \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_2 \theta_1} & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_2^2} & \cdots & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_2 \theta_d} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_d \theta_1} & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_d \theta_2} & \cdots & \frac{\partial^2 f(\boldsymbol{\theta})}{\partial \theta_d^2} \end{bmatrix}$$

Recap - Offline learning

When doing offline learning, we typically use a batch of data $x_{1:n} = x_1, x_2, \ldots, x_n$ and optimize functions of the form:

$$f(\boldsymbol{\theta}) = f(\boldsymbol{\theta}, x_{1:n}) = \frac{1}{n} \sum_{i=1}^{n} f(\boldsymbol{\theta}, x_i) \approx \int f(\boldsymbol{\theta}, \boldsymbol{x}) p(\boldsymbol{x}) dx$$

The gradient is then:

$$g(\boldsymbol{\theta}) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}) = \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, x_i)$$

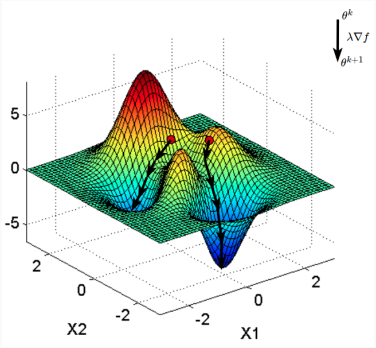
Gradient Descent

Gradient Descent Algorithm

Gradient descent or steepest descent is an *iterative* algorithm for optimization which has the following *update* form:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \boldsymbol{g}^{(k)} = \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \nabla f(\boldsymbol{\theta}^{(k)})$$

where k is the step in the algorithm, $g^{(k)} = g(\boldsymbol{\theta}^{(k)})$ - gradient at step k, $\lambda^{(k)}$ is the learning rate or step size



Gradient Descent Derivation

Function f is our *loss function* $L(\boldsymbol{\theta})$ ($J(\boldsymbol{\theta})$ notation also found often in literature)
Objective: find a *small change* $\Delta\boldsymbol{\theta}$ in parameters $\boldsymbol{\theta}$ that minimizes the value of L

$$\arg\min_{\Delta\boldsymbol{\theta}} L(\boldsymbol{\theta} + \Delta\boldsymbol{\theta}) \approx \arg\min_{\Delta\boldsymbol{\theta}} \left[L(\boldsymbol{\theta}) + \Delta\boldsymbol{\theta} \frac{\partial L}{\partial \boldsymbol{\theta}} \right]$$
$$s.t. \ \|\Delta\boldsymbol{\theta}\| \leq \epsilon$$

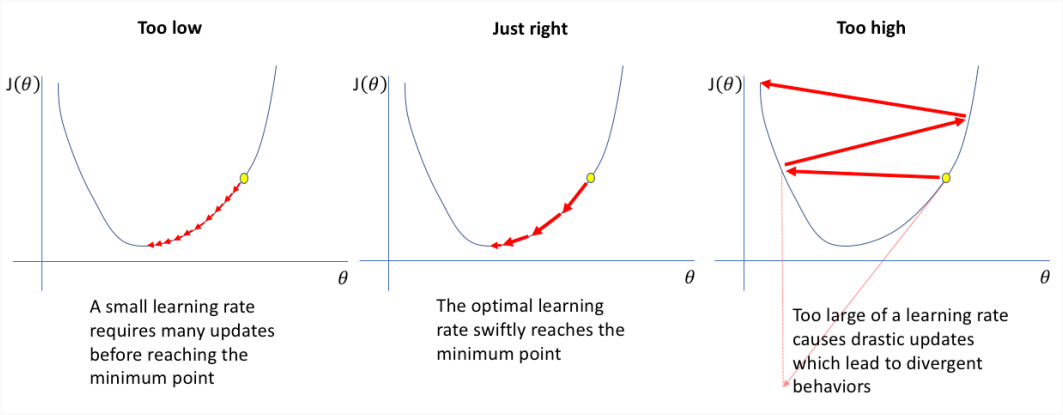
Constrained optimization problem $\longrightarrow$ *Lagrange multipliers*

$$\arg\min_{\Delta\boldsymbol{\theta}} L(\boldsymbol{\theta}) + \Delta\boldsymbol{\theta} \frac{\partial L}{\partial \boldsymbol{\theta}} + \eta \Delta\boldsymbol{\theta} I \Delta\boldsymbol{\theta}^T$$

Solving for $\Delta\boldsymbol{\theta}$ in the optimization gives us:

$$\Delta\boldsymbol{\theta} = \boldsymbol{\theta}^{(k+1)} - \boldsymbol{\theta}^{(k)} = -\frac{1}{2\eta} \frac{\partial L}{\partial \boldsymbol{\theta}} \implies \boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \lambda \frac{\partial L}{\partial \boldsymbol{\theta}}$$

How to choose the learning rate?



Source: https://www.jeremyjordan.me/nn-learning-rate/

Second Order Optimization

Newton's algorithm

The most basic second-order optimization algorithm.
Has updates of the form:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - H_K^{-1} g^{(k)}$$

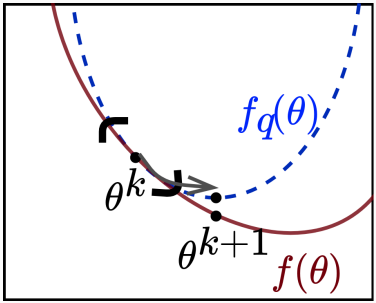
Algorithm derived from *second-order* Taylor series approx. of $L(\boldsymbol{\theta})$ around $\boldsymbol{\theta}^{(k)}$:

$$L_{quad}(\boldsymbol{\theta}) = L(\boldsymbol{\theta}^{(k)}) + g^{(k)^T}(\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)}) + \frac{1}{2}(\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)})^T H_K(\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)})$$

Differentiate, equate to 0 and solve for $\boldsymbol{\theta}$:

$$\nabla L(\boldsymbol{\theta}) = 0 + g^{(k)} + H_K(\boldsymbol{\theta} - \boldsymbol{\theta}^{(k)}) = 0$$

$$\implies \boldsymbol{\theta} = \boldsymbol{\theta}^{(k)} - H_K^{-1} g^{(k)}$$



Second Order Optimization - issues

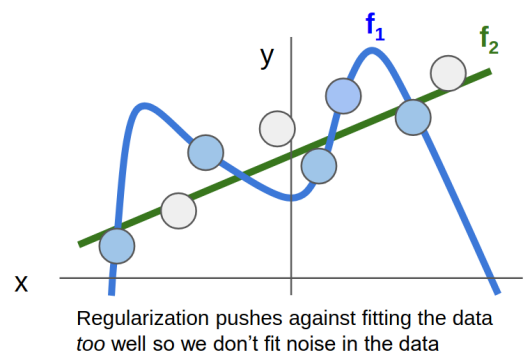
To compute the direction of descent $d^{(k)} = -\boldsymbol{H}_K^{-1} g^{(k)}$ we need to *invert* the Hessian matrix H_K, *at each iteration!*

An efficient and popular way to circumvent this issue is to use the Conjugate Gradient method to solve the linear system of equations $\boldsymbol{H}_K d^{(k)} = -g^{(k)}$ for $d^{(k)}$

Regularization

Regularization: prefer simpler models

Occam's Razor: Among multiple competing hypotheses, the simplest is the best, William of Ockham 1285-1347



Regularization: general form

- The loss term $L(\boldsymbol{\theta})$ can be decomposed into a data loss component and a regularization (parameter norm) component

$$L(\boldsymbol{\theta}) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, \boldsymbol{\theta}), y_i)}_{\text{Data loss: model predictions should match training data}} + \underbrace{\alpha R(\boldsymbol{\theta})}_{\text{Regularization: Prevent model from doing } \textit{too} \text{ well on training data}}$$

- L2 regularization: $R(\boldsymbol{\theta}) = \sum_i \theta_i^2$
- L1 regularization: $R(\boldsymbol{\theta}) = \sum_i |\theta_i|$
- α - regularization **strength** (hyperparameter)

Regularization: L2

$$L(\boldsymbol{\theta}) = L_{\text{data}}(\boldsymbol{\theta}) + \frac{\alpha}{2}\|\boldsymbol{\theta}\|_2^2$$

When taking the gradient:

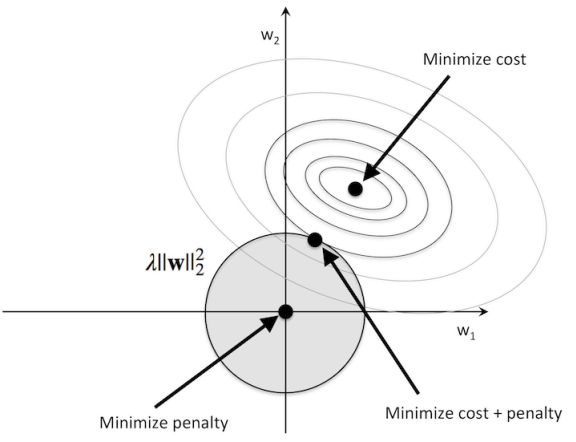
$$\nabla L(\boldsymbol{\theta}) = \nabla L_{\text{data}}(\boldsymbol{\theta}) + \alpha \cdot \|\boldsymbol{\theta}\|_2$$

Parameter updates in gradient descent steps become:

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^t - \lambda \nabla L(\boldsymbol{\theta}^t) - \lambda \cdot \alpha \cdot \boldsymbol{\theta}^t$$

Regularization: L2

Visualization for L_2 regularization



Regularization: L1

$$L(\boldsymbol{\theta}) = L_{\text{data}}(\boldsymbol{\theta}) + \alpha \cdot \|\boldsymbol{\theta}\|_1$$

When taking the gradient:

$$\nabla L(\theta) = \nabla L_{\text{data}}(\boldsymbol{\theta}) + \alpha \cdot \frac{\partial \|\boldsymbol{\theta}\|_1}{\partial \boldsymbol{\theta}}$$

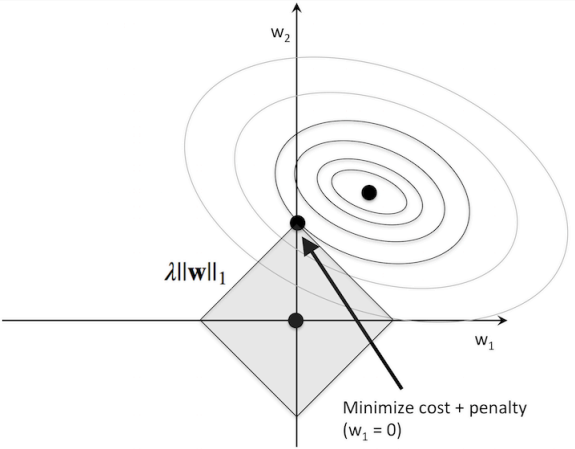
Parameter updates in gradient descent steps become:

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^t - \lambda \nabla L(\boldsymbol{\theta}^t) - \lambda \cdot \alpha, \text{ if } \boldsymbol{\theta}^t > 0$$

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^t - \lambda \nabla L(\boldsymbol{\theta}^t) + \lambda \cdot \alpha, \text{ if } \boldsymbol{\theta}^t < 0$$

Regularization L1

Visualization for L_1 regularization



Regularization: Weight Decay

Weight Decay: a penalty term (w_d) applied to the parameters when making a **parameter update step**

$$\boldsymbol{\theta}^{t+1} \leftarrow \boldsymbol{\theta}^t - \lambda \nabla L(\boldsymbol{\theta}^t) - \lambda \cdot w_d \cdot \boldsymbol{\theta}^t$$

- In typical Gradient Descent, L2 regularization and weight decay look identical, but there is a **major difference**:
  - **L2 regularization** is applied to the **loss function**, i.e. **before** taking gradients
  - **Weight Decay** is applied in the **parameter update term**, i.e. **after** the gradient of the loss has been computed

SGD

Stochastic Gradient Descent

The loss function that helps us compute the gradient with respect to model parameters $\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$ also depends on the observed data instances.

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}) = \underbrace{\int \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}, \boldsymbol{\theta}) p(\boldsymbol{x}) d\boldsymbol{x}}_{\mathbb{E}[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}, \boldsymbol{\theta})]} \overset{\text{want}}{=} 0$$

$$\begin{aligned}
\boldsymbol{\theta}^{(k+1)} &= \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \mathbb{E}\left[\nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}, \boldsymbol{\theta})\right] \\
&\approx \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \frac{1}{n} \sum_{i=1}^{n} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}^{(i)}, \boldsymbol{\theta}) \\
&\approx \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}^{(k)}, \boldsymbol{\theta}^{(k)})
\end{aligned}$$

The algorithm is *stochastic* because it only "looks" at a subset (or even instance) of the data to estimate the true gradient.

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \lambda^{(k)} \mathbb{E}\left[\nabla L\right] + \lambda^{(k)} \underbrace{\left[\mathbb{E}\left[\nabla L\right] - \nabla L(\boldsymbol{x}^{(k)}, \boldsymbol{\theta}^{(k)})\right]}_{\text{noise}}$$

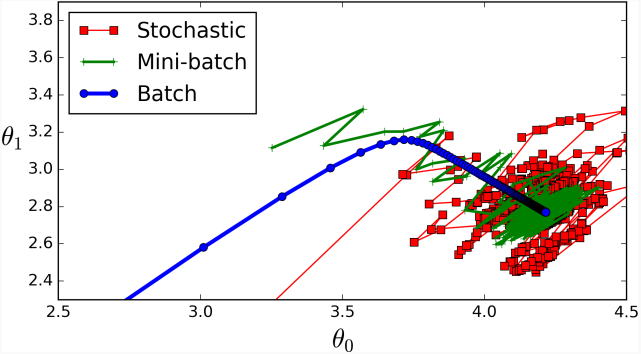
If $\lambda^{(k)}$ is a series that converges under variance of the noise, then the noise is bounded and the algorithm converges

SGD and Batches

Batch $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \lambda \frac{1}{N} \sum_{i=1}^{N} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}^{(i)}, \boldsymbol{\theta}^{(k)})$

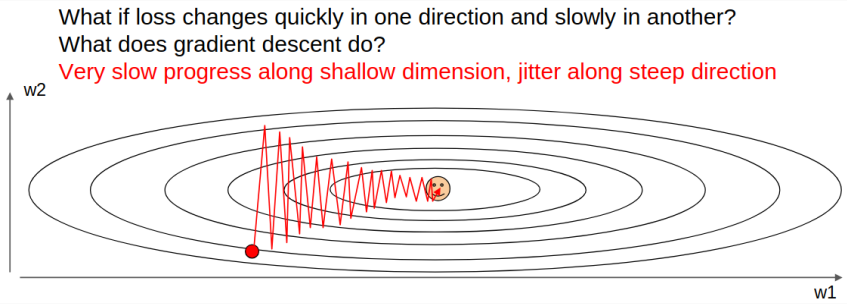
Stochastic / Online $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \lambda \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}^{(k)}, \boldsymbol{\theta}^{(k)})$

Mini-batch $\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} + \lambda \frac{1}{B} \sum_{i=1}^{B} \nabla_{\boldsymbol{\theta}} L(\boldsymbol{x}^{(i)}, \boldsymbol{\theta}^{(k)})$



SGD: Issues / Challenges

- Different rates of change in loss value depending on different dimensions of the parameter space

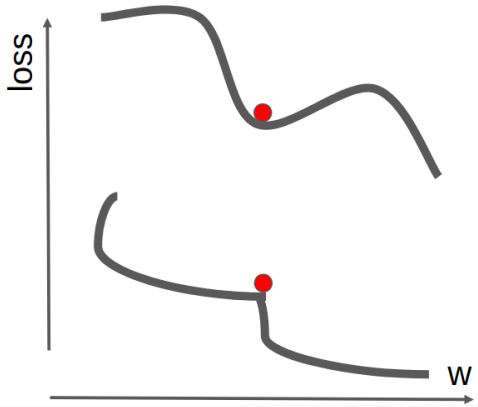


SGD: Issues / Challenges

- Local minima or saddle points of the loss function

What if the loss function has a **local minima** or **saddle point**?

Zero gradient, gradient descent gets stuck



SGD: Issues / Challenges

- **Local minima** or **saddle points** of the loss function

saddle point in two dimension

$$f(x, y) = x^2 - y^2$$

$$\frac{\partial}{\partial x}(x^2 - y^2) = 2x \rightarrow 2(0) = 0$$

$$\frac{\partial}{\partial y}(x^2 - y^2) = -2y \rightarrow -2(0) = 0$$

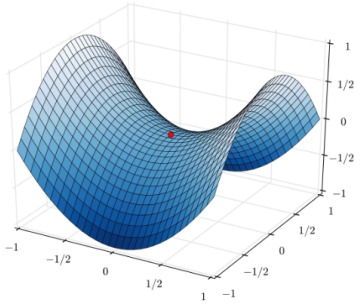


Image source: https://en.wikipedia.org/wiki/Saddle_point

SGD: Issues / Challenges

- Gradients come from mini-batches, so they could be noisy
- Gradients will be larger if input to a layer is un-normalized (un-normalized input $\Rightarrow$ larger activations $\Rightarrow$ larger loss values $\Rightarrow$ larger gradient values $\Rightarrow$ increase chance of unstable updates)
- SGD influenced by correlated inputs in batches (tendency to overfit the batch) $\Rightarrow$ increase in instability of updates

SGD: Practical concern

Choosing a proper learning rate:

- Define the *rate* of annealing / *scheduled reducing* of the learning rate

- Self-adjusting learning rate: not all parameters need to be updated with the same learning rate (e.g. for rarer or more important features we want a bigger update in that direction)

Gradient Descent Optimization Algorithms

Momentum



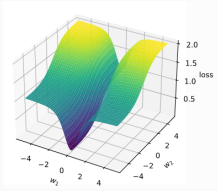


Figure 2: SGD without momentum (left) and with momentum (right). Source: DataScience Deep Dive Blog: Optimizations of Gradient Descent

Intuition: Momentum accelerates SGD in relevant direction and dampens oscillations

Let $\Delta\boldsymbol{\theta}_t$ be the update step of parameters $\boldsymbol{\theta}$ at step t

$$\Delta\boldsymbol{\theta}_t = \gamma\Delta\boldsymbol{\theta}_{t-1} + \lambda\nabla_{\boldsymbol{\theta}}L(\boldsymbol{\theta})$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \Delta\boldsymbol{\theta}_t$$

γ is usually set to 0.9.

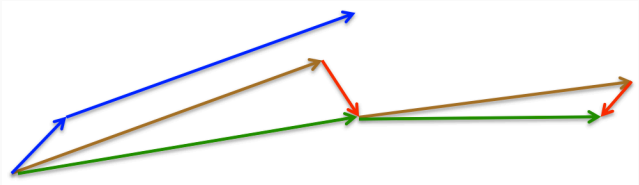
Nesterov Accelerated Gradient

Intuition: we want to give Momentum a sense of ”when to slow down” before the hill slopes up again.

Compute gradient w.r.t future estimated parameters $\boldsymbol{\theta} - \gamma \Delta \boldsymbol{\theta}_{t-1}$.

$$\Delta \boldsymbol{\theta}_t = \gamma \Delta \boldsymbol{\theta}_{t-1} + \lambda \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta} - \gamma \Delta \boldsymbol{\theta}_{t-1})$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \Delta \boldsymbol{\theta}_t$$



Momentum update (blue vectors) compared to NAG update (brown + red vectors). Source G. Hinton Lecture 6c, CSC321

Adagrad

Intuition: **adapt** learning rate of **individual parameters**, depending on parameter importance.

- smaller updates (lower learning rate) for frequently occurring features
- larger updates (higher learning rate) for infrequent ones
- suited for sparse data (e.g. object classification in video, word embeddings)

$$Notation: g_{t,i} = \nabla L_{\boldsymbol{\theta}}(\theta_i^{(t)})$$

$$\theta_i^{(t+1)} = \theta_i^{(t)} - \frac{\lambda}{\sqrt{G_{t,ii} + \epsilon}} g_{t,i}$$

the diagonal = sum of squares of past gradients (up to step t) for parameter θ_i.

$G_t \in \mathbb{R}^{d \times d}$ - diagonal matrix. Element i, i on

Adadelta

Intuition: Adagrad's weakness - accumulation of squared gradients $\implies$ learning rate shrinks $\implies$ no new knowledge updates

Adadelta solution: restrict past gradients to a window w + compute a decaying running average at each time step.

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2, \text{where } E[g^2]_t \text{ is the running average at step t.}$$

$$\Delta \boldsymbol{\theta}_t = \frac{\lambda}{\sqrt{E[g^2]_t + \epsilon}} \odot g_t = \frac{\lambda}{RMS[g]_t} \odot g_t$$
$$\boldsymbol{\theta^{(t+1)}} = \boldsymbol{\theta^{(t)}} - \Delta \boldsymbol{\theta}_t$$

Adadelta - contd

Note: theoretically, the update should have the same hypothetical units as the parameter

Adadelta solution: define an exponential decay of *squared parameter updates*

$$E[\Delta\boldsymbol{\theta}^2]_t = \gamma E[\Delta\boldsymbol{\theta}^2]_{t-1} + (1-\gamma)\Delta\boldsymbol{\theta}_t^2$$

$$\begin{aligned} RMS[\Delta\boldsymbol{\theta}]_t &= \sqrt{E[\Delta\boldsymbol{\theta}^2]_t + \epsilon} \\ \Delta\boldsymbol{\theta}_t &= \frac{RMS[\Delta\boldsymbol{\theta}]_{t-1}}{RMS[g]_t} g_t, \text{where } RMS[\Delta\boldsymbol{\theta}]_{t-1} \text{ replaces } \lambda \\ \boldsymbol{\theta}^{(t+1)} &= \boldsymbol{\theta}^{(t)} - \Delta\boldsymbol{\theta}_t \end{aligned}$$

RMSProp

Developed by Geoffrey Hinton in Lecture 6e of Coursera Class.

Equivalent to Adadelta, without the exponential decay of squared parameter updates

$$E[g^2]_t = 0.9E[g^2]_{t-1} + 0.1g_t^2$$

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\lambda}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

where $\lambda = 0.001$ is considered a good default value

Adam

Adaptive Moment Estimation (Adam) $\approx$ Adadelta with momentum.

- store exponentially decaying average of squared gradients v_t (second moment - variance)
- store also exponentially decaying average of past gradients m_t (first moment - mean), similar to momentum

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

m_t and v_t are biased towards 0 at initial time steps (especially if β_1, β_2 are close to 1)
$\implies$ bias-correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$
$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Adam update rule:

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \frac{\lambda}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

where default indicated values are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$

Adam

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Momentum

Bias correction

AdaGrad / RMSProp

Bias correction for the fact that first and second moment estimates start at zero

Adam with beta1 = 0.9,
beta2 = 0.999, and learning_rate = 1e-3 or 5e-4
is a great starting point for many models!

Kingma and Ba, "Adam: A method for stochastic optimization", ICLR 2015

AdamW

How does regularization (e.g. L2) affect Adam?

Standard Adam computes L2 here

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

Used during moment calculations!

AdamW

How does regularization (e.g. L2) affect Adam?

```
first_moment = 0
second_moment = 0
for t in range(1, num_iterations):
  dx = compute_gradient(x)
  first_moment = beta1 * first_moment  + (1 - beta1) * dx
  second_moment = beta2 * second_moment + (1 - beta2) * dx * dx
  first_unbias = first_moment / (1 - beta1 ** t)
  second_unbias = second_moment / (1 - beta2 ** t)
  x -= learning_rate * first_unbias / (np.sqrt(second_unbias) + 1e-7))
```

AdamW (Weight Decay) adds term here

Computed after the moments!

AdamW

How does regularization (e.g. L2) affect Adam?

Algorithm 2 Adam with L_2 regularization and Adam with weight decay (AdamW)

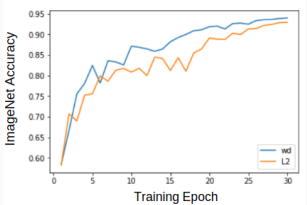
given $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}, w \in \mathbb{R}$
initialize time step $t \leftarrow 0$, parameter vector $\boldsymbol{x}_{t=0} \in \mathbb{R}^n$, first moment vector $\boldsymbol{m}_{t=0} \leftarrow \boldsymbol{0}$, second moment vector $\boldsymbol{v}_{t=0} \leftarrow \boldsymbol{0}$, schedule multiplier $\eta_{t=0} \in \mathbb{R}$
repeat
  $t \leftarrow t + 1$
  $\nabla f_t(\boldsymbol{x}_{t-1}) \leftarrow$ SelectBatch$(\boldsymbol{x}_{t-1})$ ▷ select batch and return the corresponding gradient
  $\boldsymbol{g}_t \leftarrow \nabla f_t(\boldsymbol{x}_{t-1}) + w\boldsymbol{x}_{t-1}$
  $\boldsymbol{m}_t \leftarrow \beta_1 \boldsymbol{m}_{t-1} + (1 - \beta_1)\boldsymbol{g}_t$ ▷ here and below all operations are element-wise
  $\boldsymbol{v}_t \leftarrow \beta_2 \boldsymbol{v}_{t-1} + (1 - \beta_2)\boldsymbol{g}_t^2$
  $\hat{\boldsymbol{m}}_t \leftarrow \boldsymbol{m}_t / (1 - \beta_1^t)$ ▷ β_1 is taken to the power of t
  $\hat{\boldsymbol{v}}_t \leftarrow \boldsymbol{v}_t / (1 - \beta_2^t)$ ▷ β_2 is taken to the power of t
  $\eta_t \leftarrow$ SetScheduleMultiplier(t) ▷ can be fixed, decay, or also be used for warm restarts
  $\boldsymbol{x}_t \leftarrow \boldsymbol{x}_{t-1} - \eta_t \left(\alpha \hat{\boldsymbol{m}}_t / (\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon) + w\boldsymbol{x}_{t-1} \right)$
until *stopping criterion is met*
return optimized parameters $\boldsymbol{x}_t$

Taken from "Fixing Weight Decay Regularization in Adam" by Ilya Loshchilov, Frank Hutter.

- Adam with L2 leads to an update of the form

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^t - \lambda \cdot \frac{\beta_1 \cdot m_t + (1 - \beta_1) \cdot (\nabla L_t + w_d \cdot \boldsymbol{\theta}^t)}{\sqrt{v_t} + \epsilon}$$

- $\Longrightarrow$ can encounter situation where a weight whose gradients are large or change a lot (high v_t) is regularized *less*, than one that is small or changes slowly.



Source: https://www.fast.ai/posts/2018-07-02-adam-weight-decay.html

Adam: other variants

Research into variations of Adam that address different challenges in neural net training:

- Ensuring stability of learning rates by:
  - maintaining a maximum of past squared gradients (e.g AMSGrad) or bounding learning rates to specific limits (e.g. AdaBound)
  - adjusting the variance of adaptive learning rates, especially at the beginning of training (e.g Rectified Adam)

Adam: other variants

Research into variations of Adam that address different challenges in neural net training:

- Ensuring stability of learning rates by:
  - maintaining a maximum of past squared gradients (e.g AMSGrad) or bounding learning rates to specific limits (e.g. AdaBound)
  - adjusting the variance of adaptive learning rates, especially at the beginning of training (e.g Rectified Adam)
- Adapt regularization: incorporate adaptive regularization into ADAM to control regularization strength dynamically (e.g. AdaRDA)

Adam: other variants

Research into variations of Adam that address different challenges in neural net training:

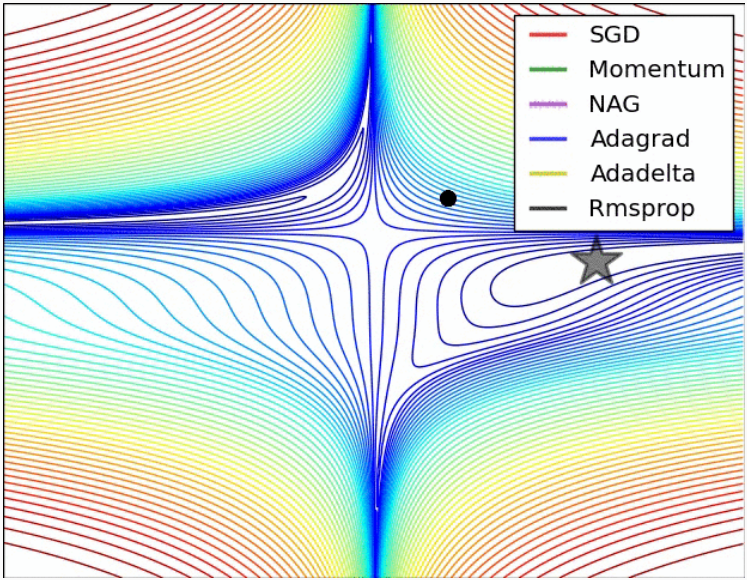
- Ensuring stability of learning rates by:
  - maintaining a maximum of past squared gradients (e.g AMSGrad) or bounding learning rates to specific limits (e.g. AdaBound)
  - adjusting the variance of adaptive learning rates, especially at the beginning of training (e.g Rectified Adam)
- Adapt regularization: incorporate adaptive regularization into ADAM to control regularization strength dynamically (e.g. AdaRDA)
- Handle sparse gradients (e.g. encountered in NLP and RL tasks): modify the second-moment estimator by excluding some gradient information to avoid bias (e.g. AdaShift)

Adam: other variants

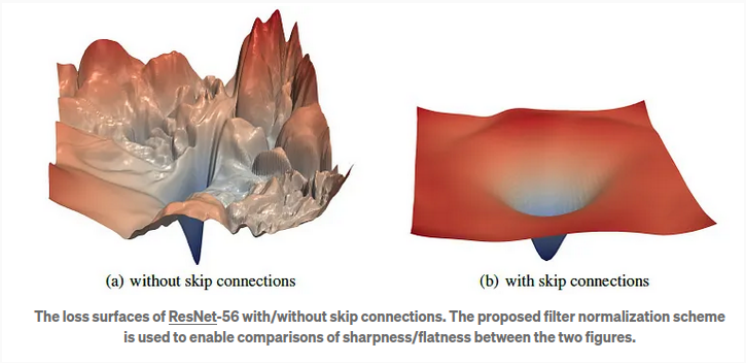
Research into variations of Adam that address different challenges in neural net training:

- Ensuring stability of learning rates by:
  - maintaining a maximum of past squared gradients (e.g AMSGrad) or bounding learning rates to specific limits (e.g. AdaBound)
  - adjusting the variance of adaptive learning rates, especially at the beginning of training (e.g Rectified Adam)
- Adapt regularization: incorporate adaptive regularization into ADAM to control regularization strength dynamically (e.g. AdaRDA)
- Handle sparse gradients (e.g. encountered in NLP and RL tasks): modify the second-moment estimator by excluding some gradient information to avoid bias (e.g. AdaShift)
- Large batch training: extends Adam by applying layer-wise learning rates (e.g. LAMB)

Visualization of GD optimization algorithms



Loss Surfaces in practice



(a) without skip connections

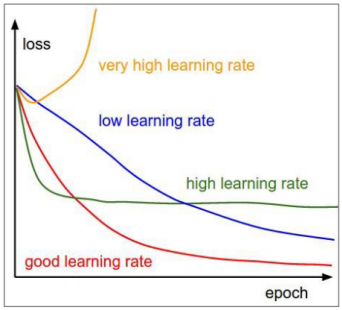
(b) with skip connections

The loss surfaces of ResNet-56 with/without skip connections. The proposed filter normalization scheme is used to enable comparisons of sharpness/flatness between the two figures.

- Loss surfaces in practice are a convoluted combination of ”pathological loss curves” (e.g. narrow valleys, steep local minima, plateaus)
- One objective of **model architectures** and **normalization techniques** is to *smoothen* the loss surface, such that theoretical considerations of studied optimizers apply

Optimization: Learning rate schedules

- SGD, SGD+Momentum, RMSProp, Adam, AdamW all have **learning rate** as a **hyperparameter**
- Q: how to choose an appropriate learning rate?

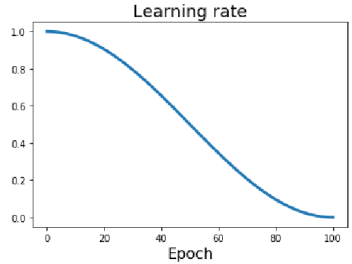


Q: Which one of these learning rates is best to use?

Optimization: Learning rate schedules

- Learning rates can be adjusted using a **schedule** based on the number of training epochs

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

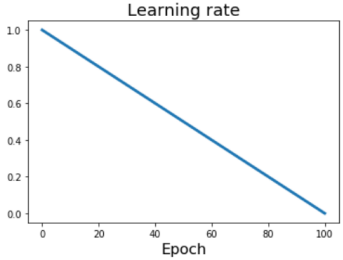
α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Loshchilov and Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts", ICLR 2017
Radford et al, "Improving Language Understanding by Generative Pre-Training", 2018
Feichtenhofer et al, "SlowFast Networks for Video Recognition", arXiv 2018
Child at al, "Generating Long Sequences with Sparse Transformers", arXiv 2019

Optimization: Learning rate schedules

- Learning rates can be adjusted using a **schedule** based on the number of training epochs

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

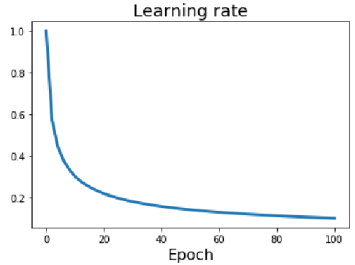
α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Devlin et al, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", 2018

Optimization: Learning rate schedules

- Learning rates can be adjusted using a **schedule** based on the number of training epochs

Learning Rate Decay



Step: Reduce learning rate at a few fixed points. E.g. for ResNets, multiply LR by 0.1 after epochs 30, 60, and 90.

Cosine: $\alpha_t = \frac{1}{2}\alpha_0 \left(1 + \cos(t\pi/T)\right)$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

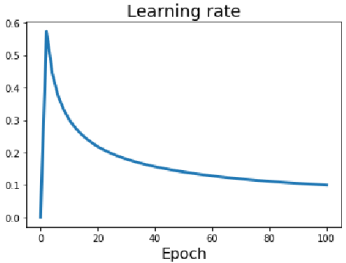
α_0 : Initial learning rate
α_t : Learning rate at epoch t
T : Total number of epochs

Vaswani et al, "Attention is all you need", NIPS 2017

Optimization: Learning rate schedules

- Learning rates can be adjusted using a **schedule** based on the number of training epochs

Learning Rate Decay: Linear Warmup



High initial learning rates can make loss explode; linearly increasing learning rate from 0 over the first ~5,000 iterations can prevent this.

Empirical rule of thumb: If you increase the batch size by N, also scale the initial learning rate by N

Goyal et al, "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour", arXiv 2017

Optimization - other practical suggestions

Optimization: good practices

- Normalize the input space:
  - Shift inputs to have a *zero mean* over the whole dataset
  - Scale the inputs to have unit variance
- Decorrelate input components
  - For linear layers, we get a big win by decorrelating each component of the input from the other components (e.g. use PCA to remove components with small eigenvalues, divide remaining principal components by the sqrt of their eigenvalues)
- Initialization of weights should break symmetry (e.g. Xavier initialization, He initialization)
- Aim for class balance in your mini-batches

Optimization: good practices

- Adam(W) - good default choice in many cases; it often works ok even with constant learning rate
- SGD + Momentum can outperform Adam but may require more tuning of the learning rate and a learning rate schedule
- For small datasets where **your available hardware** (memory constraints) can afford **full batch** updates – try to look beyond 1st order optimization (e.g. LBFGS, conjugate gradient)

- Empirical rule of thumb: If you increase the batch size by N, also scale the initial learning rate by N

Optimization: additional aspects

- Shuffling and Curriculum Learning
  - Shuffle training data after each epoch, to avoid biasing the optimization algorithm
  - **Curriculum Learning**: for some problems (e.g. involving a temporal dimension), training using progressively harder examples helps

- Batch Normalization
  - Common practice: normalize inputs to zero mean and unit variance
  - For deep networks, intermediary layer inputs may vary significantly during training (covariate shift) $\implies$ *normalize* intermediary layer inputs

Optimization: additional aspects

- **Early stopping** (you should do this)
  - Monitor error on *validation set* and **stop** (with some patience, i.e. number of gradient update steps) if validation error does not improve enough
  - Alternative: train until end of epochs, but keep a **best model** (by validation loss) record

- **Gradient Noise**
  - Used to make training more robust to poor initialization, or when having deep and complex networks (highly irregular error function - escape local minima)
  - Gradient modification:

$$g_{t,i} = g_{t,i} + N(0, \sigma_t^2)$$
$$\sigma_t^2 = \frac{\eta}{(1+t)^\gamma}$$

Optimization: additional aspects

- Consider **Dropout** as a regularization method:
  - in each forward pass, randomly set some neurons to zero
  - probability of dropping is a hyperparameter; values between [0.1, 0.5] are common
- Stochastic Weight Averaging (SWA)
  - Effective, low computational overhead method to exploit SGD behavior in ”flattened” loss function space (when approaching convergence)
  - https://pytorch.org/blog/pytorch-1.6-now-includes-stochastic-weight-averaging/

References

References

Credits for slides to lectures by Nando de Freitas, Geoffrey Hinton and article by Sebastian Ruder.

- Nando de Freitas. Machine Learning. Lecture 5 - Optimization
- Sebastian Ruder (2016). An overview of gradient descent optimisation algorithms. arXiv preprint arXiv:1609.04747.
- Geoffrey Hinton. Neural Networks for Machine Learning. Lecture 6
- Fei-Fei Li, Ehsan Adeli, Stanford CS231n Spring 2024 class, Lectures 3 and 4